

CSE489: Selected Topics in Data Science

Extractive Summarization

Project Documentation

Yehia Mahmoud Lotfy Sakr	21P0166
Hana Mohamed Mohamed	21P0162

Contents

Project Overview 3
Project Objectives 3
Phase 1: Setup, Utilities, and Data Preparation 4
Step 1: Setup and Environment 4
Step 2: Hyperparameters and Global State (The Config Class) 5
Step 3: Core NLP Utilities 6
Step 4: Data Loading and Preparation 7
Phase 2: Feature Engineering 8
Step 5: Feature Engineering and Embeddings 8
Phase 3: Model Training and Baseline Evaluation 10
Step 6: Baseline Model Development 10
Step 7: Evaluation Metrics 11
Step 8: Example Output and Metric Saving 13
Step 9: Performance Visualization 14
Phase 4: Advanced Features & Interactive UI 16
Step 9: Custom Testing Interface 16
Step 10: Ultimate Summarization Dashboard 17
Phase 5: Baselines and Alternative Architectures 19
Step 11: Lead-3 Baseline Comparison 19
Step 12: Alternative Architecture (MiniLM + XGBoost) 21
Step 13: Benchmarking Alternative Architectures 23
Conclusion 24

Project Overview

This project focuses on the development of an **Extractive Text Summarization** pipeline. Unlike abstractive summarization (which generates new sentences), extractive summarization identifies and extracts the most salient sentences from a source document to create a concise summary.

By leveraging modern transformer-based embeddings (DistilBERT and MiniLM) alongside machine learning classifiers (Logistic Regression and XGBoost), this system transforms summarization into a binary classification problem: predicting whether a specific sentence should be included in the final summary.

Project Objectives

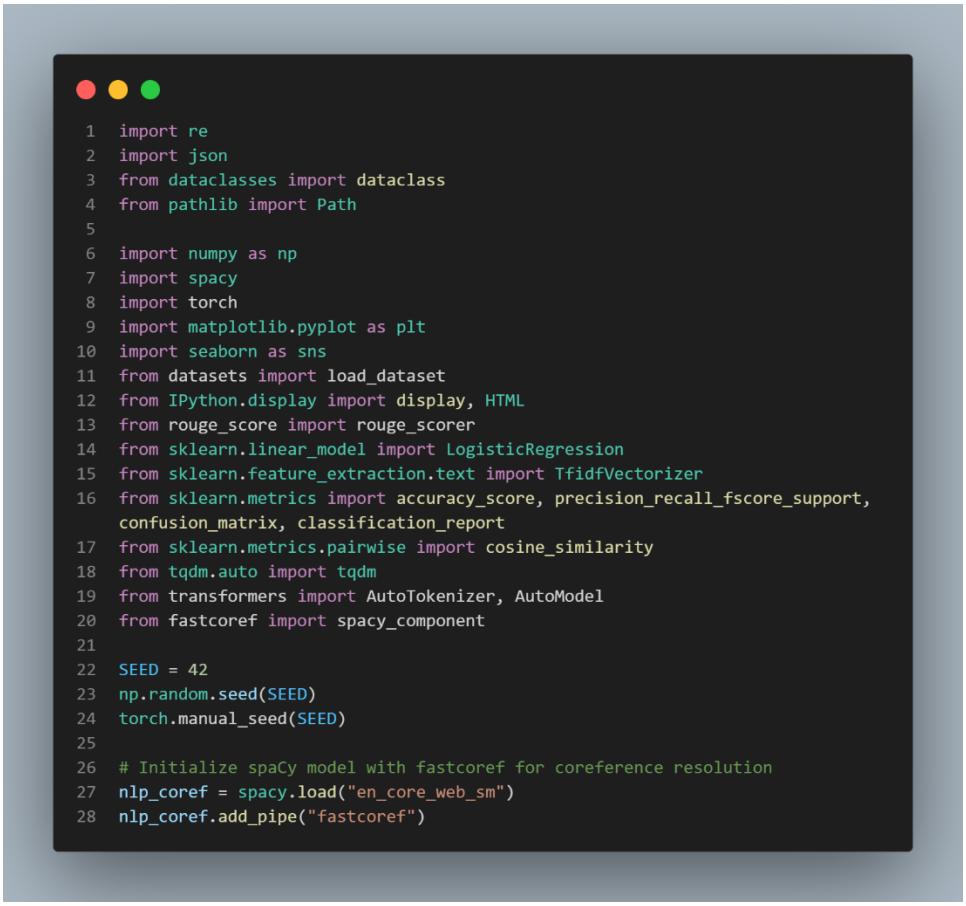
The goal is to deliver a robust, interpretable, and high-performance summarization tool with the following features:

- **Hybrid Feature Engineering**: Combines deep semantic embeddings from Transformers with handcrafted heuristic features such as sentence position, length, and TF-IDF relevance.
- **Architectural Benchmarking**: Compares a standard pipeline (DistilBERT + Logistic Regression) against a lightweight, high-speed alternative (MiniLM + XGBoost) to optimize for different hardware constraints.
- **Contextual Integrity**: Implements Coreference Resolution to resolve pronouns (e.g., 'he', 'it') to their original subjects, ensuring summaries remain coherent.
- **Information Diversity**: Utilizes the **Maximal Marginal Relevance (MMR)** algorithm to reduce redundancy, ensuring each selected sentence adds unique value.
- **Automated Metadata Extraction**: Integrates Named Entity Recognition (NER) and TF-IDF-based theme identification to provide quick insights into the 'who' and 'what' of the text.
- **Quantitative Evaluation**: Benchmarks performance against the **Lead-3 baseline** using standardized **ROUGE** metrics.
- **Interactive Interface**: Provides a dashboard for real-time experimentation with model parameters and custom text inputs.

Phase 1: Setup, Utilities, and Data Preparation

Step 1: Setup and Environment

The project environment is initialized with essential NLP and machine learning dependencies (transformers, torch, spaCy, datasets, and rouge-score). To guarantee experimental reproducibility, a global random seed (SEED = 42) is strictly enforced across numpy and torch. Furthermore, fastcoref is explicitly integrated into the standard spaCy pipeline to enable robust, neural-based pronoun resolution.



```python
import re
import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import spacy
import torch
import matplotlib.pyplot as plt
import seaborn as sns
from datasets import load_dataset
from IPython.display import display, HTML
from rouge_score import rouge_scorer
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_recall_fscore_support,
confusion_matrix, classification_report
from sklearn.metrics.pairwise import cosine_similarity
from tqdm.auto import tqdm
from transformers import AutoTokenizer, AutoModel
from fastcoref import spacy_component

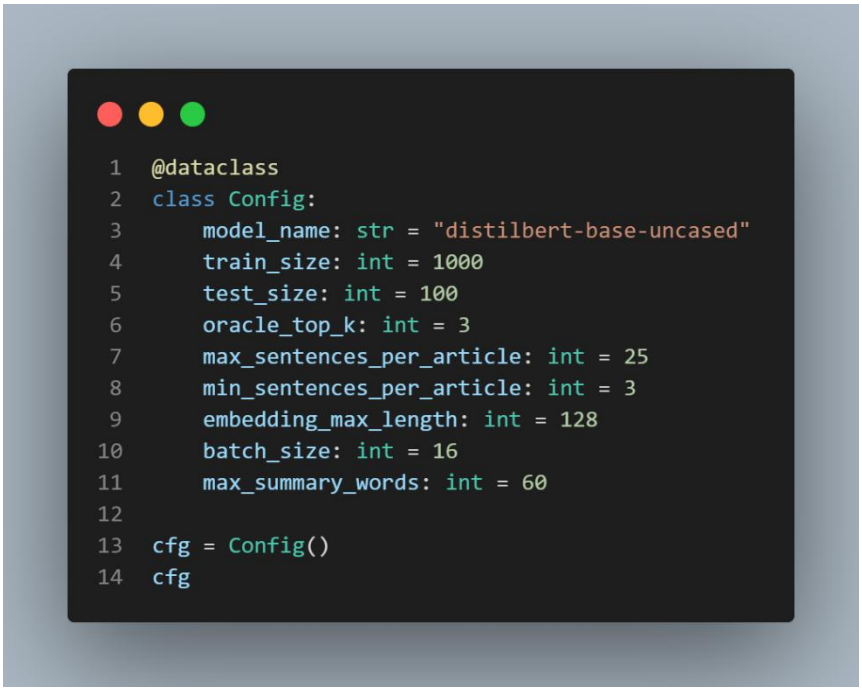
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

# Initialize spaCy model with fastcoref for coreference resolution
nlp_coref = spacy.load("en_core_web_sm")
nlp_coref.add_pipe("fastcoref")
```

Step 2: Hyperparameters and Global State (The Config Class)

To ensure the pipeline is modular and easily tunable, all global parameters are centralized within a Python @dataclass named Config. This class defines the state of the entire system:

- **model_name:** Set to "distilbert-base-uncased" for the primary embedding engine.
- **train_size (1000) & test_size (100):** Dictates the sample boundaries for rapid experimentation.
- **oracle_top_k (3):** Defines how many reference sentences constitute a "summary-worthy" ground truth.
- **min_sentences_per_article (3) & max_sentences_per_article (25):** Filters out anomalies and truncates massive documents to maintain manageable tensor dimensions.
- **embedding_max_length (128) & batch_size (16):** Controls the Transformer token limits and GPU memory utilization.
- **max_summary_words (60):** The hard limit imposed on the final generated summary.



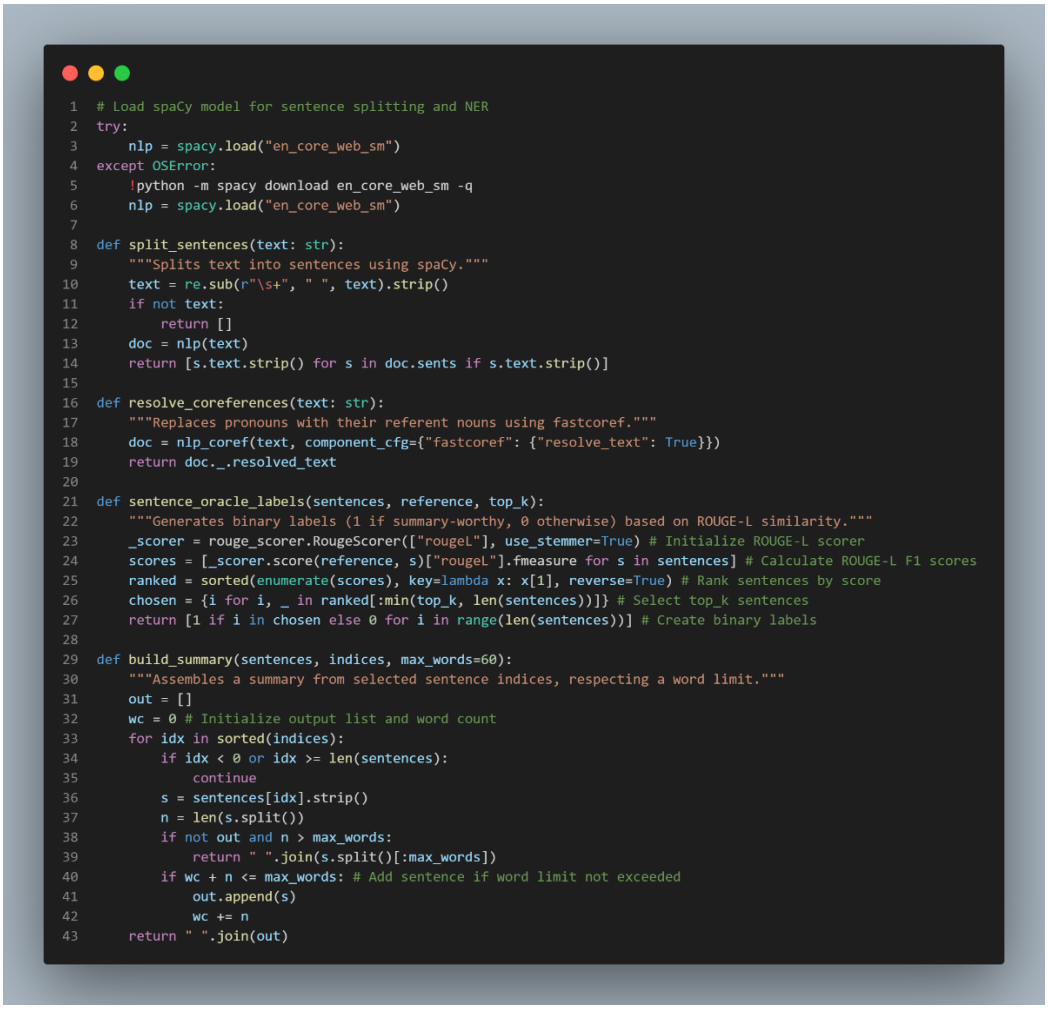
```python
@dataclass
class Config:
    model_name: str = "distilbert-base-uncased"
    train_size: int = 1000
    test_size: int = 100
    oracle_top_k: int = 3
    max_sentences_per_article: int = 25
    min_sentences_per_article: int = 3
    embedding_max_length: int = 128
    batch_size: int = 16
    max_summary_words: int = 60

cfg = Config()
cfg
```

Step 3: Core NLP Utilities

A foundational suite of helper functions acts as the preprocessing engine:

- **Sentence Splitting:** Uses spaCy to accurately parse long text blocks into distinct sentence boundaries.
- **Coreference Resolution:** Automatically maps and replaces pronouns (e.g., *he*, *it*) with their explicit entities, preserving context when sentences are extracted in isolation.
- **Oracle Label Creation:** Automates the creation of binary training labels (1 or 0). It calculates the ROUGE-L F1 similarity between every article sentence and the provided reference highlights, flagging the top K sentences as summary-worthy.
- **Summary Builder:** A dedicated constructor that stitches predicted sentences back together in their original chronological order, enforcing the max_summary_words constraint.



```python
# Load spaCy model for sentence splitting and NER
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    !python -m spacy download en_core_web_sm -q
    nlp = spacy.load("en_core_web_sm")

def split_sentences(text: str):
    """Splits text into sentences using spaCy."""
    text = re.sub(r"\s+", " ", text).strip()
    if not text:
        return []
    doc = nlp(text)
    return [s.text.strip() for s in doc.sents if s.text.strip()]

def resolve_coreferences(text: str):
    """Replaces pronouns with their referent nouns using fastcoref."""
    doc = nlp_coref(text, component_cfg={"fastcoref": {"resolve_text": True}})
    return doc._.resolved_text

def sentence_oracle_labels(sentences, reference, top_k):
    """Generates binary labels (1 if summary-worthy, 0 otherwise) based on ROUGE-L similarity."""
    _scorer = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True) # Initialize ROUGE-L scorer
    scores = [_scorer.score(reference, s)["rougeL"].fmeasure for s in sentences] # Calculate ROUGE-L F1 scores
    ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True) # Rank sentences by score
    chosen = {i for i, _ in ranked[:min(top_k, len(sentences))]} # Select top_k sentences
    return [1 if i in chosen else 0 for i in range(len(sentences))] # Create binary labels

def build_summary(sentences, indices, max_words=60):
    """Assembles a summary from selected sentence indices, respecting a word limit."""
    out = []
    wc = 0 # Initialize output list and word count
    for idx in sorted(indices):
        if idx < 0 or idx >= len(sentences):
            continue
        s = sentences[idx].strip()
        n = len(s.split())
        if not out and n > max_words:
            return " ".join(s.split()[:max_words])
        if wc + n <= max_words: # Add sentence if word limit not exceeded
            out.append(s)
            wc += n
    return " ".join(out)
```

Step 4: Data Loading and Preparation

The **CNN/DailyMail (3.0.0)** dataset is loaded via HuggingFace for training and testing. To optimize experimentation, a representative subset was sampled. Articles were filtered for minimum length requirements, and sentences were truncated to keep the pipeline manageable. Oracle labels were then mapped to each sentence using the article's provided highlights to formulate a structured training array.

```python
print("Loading CNN/DailyMail...")
ds = load_dataset("cnn_dailymail", "3.0.0")
train_rows = ds["train"].shuffle(seed=SEED).select(range(cfg.train_size))
test_rows = ds["validation"].shuffle(seed=SEED).select(range(cfg.test_size))

def make_examples(rows):
    """Prepares dataset examples with sentence splitting, truncation, and oracle labels."""
    examples = []
    for r in rows: # Process each raw article
        sents = split_sentences(r["article"]) # Split article into sentences
        if len(sents) < cfg.min_sentences_per_article:
            continue
        sents = sents[:cfg.max_sentences_per_article] # Truncate sentences
        labels = sentence_oracle_labels(sents, r["highlights"], cfg.oracle_top_k) # Generate oracle labels
        examples.append({"article": r["article"], "reference": r["highlights"], "sentences": sents, "labels": labels})
    return examples

train_examples = make_examples(train_rows)
test_examples = make_examples(test_rows)
print(f"Prepared train examples: {len(train_examples)}")
print(f"Prepared test examples: {len(test_examples)}")
```

```
Loading CNN/DailyMail...
README.md: 15.6k/? [00:00<00:00, 775kB/s]
3.0.0/train-00000-of-00003.parquet: 100% 257M/257M [00:01<00:00, 438MB/s]
3.0.0/train-00001-of-00003.parquet: 100% 257M/257M [00:05<00:00, 320MB/s]
3.0.0/train-00002-of-00003.parquet: 100% 259M/259M [00:05<00:00, 47.5MB/s]
3.0.0/validation-00000-of-00001.parquet: 100% 34.7M/34.7M [00:00<00:00, 173MB/s]
3.0.0/test-00000-of-00001.parquet: 100% 30.0M/30.0M [00:00<00:00, 86.2MB/s]
Generating train split: 100% 287113/287113 [00:08<00:00, 24010.30 examples/s]
Generating validation split: 100% 13368/13368 [00:03<00:00, 2836.92 examples/s]
Generating test split: 100% 11490/11490 [00:00<00:00, 45686.85 examples/s]
Prepared train examples: 1000
Prepared test examples: 100
```

Phase 2: Feature Engineering

Step 5: Feature Engineering and Embeddings

To transform text into machine-readable tensors, a hybrid feature engineering approach combines deep learning with traditional statistics:

- **Dense Contextual Vectors:** Sentences are processed in batches through the **distilbert-base-uncased** model. A mean-pooling operation is applied to the last hidden states (using the attention mask to ignore padding), yielding dense, high-dimensional semantic vectors.

Deep semantic embeddings (like DistilBERT) are excellent at capturing the contextual meaning of a sentence, but they lack structural and statistical awareness of the document as a whole. Injecting handcrafted features gives the classifier critical "meta-knowledge" about the text.

- **Statistical Heuristics:** Handcrafted metrics are calculated for every sentence, including:
    - ***Relative Position:*** (Index / Total Sentences), as news articles heavily front-load important data.
    - ***Word Count:*** Capturing sentence density.
    - ***TF-IDF Relevance:*** Summing term-frequency vectors to identify keyword density.
- **Feature Concatenation:** The 1D statistical arrays are horizontally stacked onto the 2D DistilBERT embedding matrices, resulting in a single, robust feature vector for the classifier.

```python
class Embedder:
    """Handles loading a transformer model and generating sentence embeddings."""
    def __init__(self, model_name, max_length):
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.tok = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name).to(self.device)
        self.model.eval()
        self.max_length = max_length

    @torch.no_grad()
    def encode(self, texts, batch_size=16):
        """Encodes texts into embeddings using mean pooling."""
        all_vecs = []
        for i in tqdm(range(0, len(texts), batch_size), desc="Embedding Batches", leave=False): # Process texts in batches
            batch = texts[i:i+batch_size]
            enc = self.tok(batch, padding=True, truncation=True, max_length=self.max_length, return_tensors="pt").to(self.device) # Tokenize and move to device
            h = self.model(**enc).last_hidden_state # Get hidden states
            m = enc["attention_mask"].unsqueeze(-1) # Prepare attention mask
            vec = (h * m).sum(dim=1) / m.sum(dim=1).clamp(min=1) # Mean pooling for embeddings
            all_vecs.append(vec.cpu().numpy()) # Store embeddings
        return np.vstack(all_vecs).astype(np.float32)

embedder = Embedder(cfg.model_name, cfg.embedding_max_length)

def flatten(examples):
    """Flattens examples into sentence-level features (embeddings, position, length, TF-IDF) and labels."""
    sents, labels = [], []
    extra_features = [] # [pos, length, tfidf]

    for ex in tqdm(examples, desc="Extracting Features"): # Process each example
        num_sents = len(ex["sentences"])

        vectorizer = TfidfVectorizer(stop_words='english') # Initialize TF-IDF vectorizer
        try:
            tfidf_matrix = vectorizer.fit_transform(ex["sentences"])
            tfidf_scores = np.asarray(tfidf_matrix.sum(axis=1)).ravel() # Calculate sentence TF-IDF scores
        except:
            tfidf_scores = np.zeros(num_sents)

        for i in range(num_sents):
            pos = i / (num_sents - 1) if num_sents > 1 else 0.0 # Calculate sentence position feature
            length = len(ex["sentences"][i].split()) # Calculate sentence length feature
            tfidf = tfidf_scores[i]

            extra_features.append([pos, length, tfidf])
            sents.append(ex["sentences"][i])
            labels.append(ex["labels"][i])

    X_embeds = embedder.encode(sents, batch_size=cfg.batch_size) # Encode sentences to embeddings
    X = np.hstack([X_embeds, np.array(extra_features, dtype=np.float32)]) # Combine embeddings and extra features
    y = np.array(labels, dtype=np.int64)
    return X, y
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.75kB/s]
config.json: 100% 483/483 [00:00<00:00, 51.7kB/s]
vocab.txt: 232k/? [00:00<00:00, 3.01MB/s]
tokenizer.json: 466k/? [00:00<00:00, 7.53MB/s]
model.safetensors: 100% 268M/268M [00:02<00:00, 176MB/s]
```

Phase 3: Model Training and Baseline Evaluation

Step 6: Baseline Model Development

A Logistic Regression classifier was trained on the sentence-level dataset to predict summary worthiness. Because non-summary sentences heavily outnumber summary-worthy ones, class weight balancing was applied. The model was evaluated on its sentence-classification capabilities (Accuracy, Precision, Recall, F1 Score, and Confusion Matrix).

```python
# Generate features for training and testing
print("Extracting training features...")
X_train, y_train = flatten(train_examples)
print("Extracting test features...")
X_test, y_test = flatten(test_examples)

# Train Logistic Regression classifier with balanced class weights
clf = LogisticRegression(max_iter=1000, class_weight="balanced", random_state=SEED, n_jobs=-1)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
p, r, f1, _ = precision_recall_fscore_support(y_test, y_pred, average="binary", zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print(f"Sentence Accuracy: {acc:.4f}")
print(f"Precision: {p:.4f}  Recall: {r:.4f}  F1: {f1:.4f}")
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(cm)
print("\nClassification report:\n")
print(classification_report(y_test, y_pred, digits=4))
```

```
Extracting training features...
Extracting Features: 100% 1000/1000 [00:02<00:00, 492.69it/s]
Extracting test features...
Extracting Features: 100% 100/100 [00:00<00:00, 460.30it/s]
Sentence Accuracy: 0.6502
Precision: 0.2553  Recall: 0.7967  F1: 0.3867
Confusion matrix [[TN, FP], [FN, TP]]:
[[1170  697]
 [  61  239]]

Classification report:

              precision    recall  f1-score   support

           0     0.9504    0.6267    0.7553      1867
           1     0.2553    0.7967    0.3867       300

    accuracy                         0.6502      2167
   macro avg     0.6029    0.7117    0.5710      2167
weighted avg     0.8542    0.6502    0.7043      2167
```

The Logistic Regression baseline demonstrates a classic handling of a highly imbalanced dataset (where non-summary sentences vastly outnumber summary-worthy ones).

- **High Recall (0.7967):** The model is very good at identifying the sentences that belong in the summary. A recall of nearly 80% means it successfully captures the vast majority of the "ground truth" sentences.
- **Low Precision (0.2553):** Because class weights were balanced to favor recall, the model heavily over-predicts the positive class. This results in many "false alarms" (non-summary sentences predicted as summary-worthy).
- **Overall Assessment:** In an extractive pipeline, this is actually an optimal trade-off. It is better for the classifier to over-select candidate sentences (high recall) and let downstream algorithms—like Maximal Marginal Relevance (MMR) and the strict word-count limits—filter out the redundant false positives, rather than missing key information entirely.

Step 7: Evaluation Metrics

The architecture is rigorously tested on the hold-out set. The pipeline embeds the unseen text, classifies the sentences, and constructs full summaries. Performance is evaluated on two fronts:

1. **Sentence-Level Classification:** Tracking Accuracy, Precision, Recall, and F1-Score.
2. **Summary-Level Generation:** Utilizing automated ROUGE-1, ROUGE-2, and ROUGE-L scorers to calculate lexical overlap against the human-written reference highlights.

```python
def predict_summary(ex):
    """Predicts an extractive summary using the Logistic Regression model and handcrafted features."""
    num_sents = len(ex["sentences"])
    embeddings = embedder.encode(ex["sentences"], batch_size=cfg.batch_size) # Get sentence embeddings

    # Calculate extra features matching training
    vectorizer = TfidfVectorizer(stop_words='english') # Initialize TF-IDF vectorizer
    try:
        tfidf_matrix = vectorizer.fit_transform(ex["sentences"])
        tfidf_scores = np.asarray(tfidf_matrix.sum(axis=1)).ravel() # Calculate TF-IDF scores
    except:
        tfidf_scores = np.zeros(num_sents)

    extra = []
    for i in range(num_sents):
        pos = i / (num_sents - 1) if num_sents > 1 else 0.0 # Calculate position feature
        length = len(ex["sentences"][i].split()) # Calculate length feature
        extra.append([pos, length, tfidf_scores[i]])

    X = np.hstack([embeddings, np.array(extra, dtype=np.float32)]) # Combine features
    probs = clf.predict_proba(X)[:, 1] # Get prediction probabilities
    top_idx = np.argsort(-probs).tolist()[:cfg.oracle_top_k] # Select top sentences based on probability
    return build_summary(ex["sentences"], top_idx, cfg.max_summary_words)

# Initialize ROUGE scorer
scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)
scores = {"rouge1": [], "rouge2": [], "rougeL": []}

# Calculate ROUGE scores for predicted summaries on test set
for ex in tqdm(test_examples, desc="ROUGE"):
    pred = predict_summary(ex)
    s = scorer.score(ex["reference"], pred)
    for k in scores: scores[k].append(s[k].fmeasure)

print("Average ROUGE F1 (With Position + Length + TFIDF):")
for k, v in scores.items(): print(f"{k}: {np.mean(v):.4f}")
```

```
ROUGE: 100% 100/100 [00:06<00:00, 13.69it/s]
Average ROUGE F1 (With Position + Length + TFIDF):
rouge1: 0.3846
rouge2: 0.1677
rougeL: 0.2650
```

The ROUGE scores provide a quantitative measure of lexical overlap between the generated summaries and the human references:

- **ROUGE-1 (0.3846):** Approaching ~38.5% unigram overlap is a highly respectable score for extractive summarization on the CNN/DailyMail dataset. It proves the model is capturing the core vocabulary of the ground truth.
- **ROUGE-2 (0.1677):** The drop in ROUGE-2 (bigrams) is expected in extractive summarization. Because the model is extracting full sentences rather than

generating custom phrasing, exact multi-word sequence matches with abstractive human summaries are harder to achieve.

- **ROUGE-L (0.2650):** This score indicates the longest common subsequence. A score of 26.5% shows the model is doing a decent job of preserving sentence structure and readability compared to the reference.

Step 8: Example Output and Metric Saving

To ensure qualitative tracking, the system randomly selects test articles and prints the original text, the reference summary, and the model's predicted summary side-by-side. All quantitative metrics are serialized and saved to a JSON file for future reference and reproducibility.

```python
sample = test_examples[np.random.randint(0, len(test_examples))]
pred_summary = predict_summary(sample)

# Print original article, reference summary, and predicted summary
print("ARTICLE:\n")
print(sample["article"])
print("REFERENCE SUMMARY:\n")
print(sample["reference"])
print("\nPREDICTED SUMMARY:\n")
print(pred_summary)

# Save computed metrics to a JSON file
metrics = {
    "sentence_accuracy": float(acc),
    "sentence_precision": float(p),
    "sentence_recall": float(r),
    "sentence_f1": float(f1),
    "confusion_matrix": cm.tolist(),
    "rouge1_f1": float(np.mean(scores["rouge1"])),
    "rouge2_f1": float(np.mean(scores["rouge2"])),
    "rougeL_f1": float(np.mean(scores["rougeL"])),
    "config": cfg.__dict__,
}
Path("outputs").mkdir(exist_ok=True)
with open(Path("outputs") / "notebook_metrics.json", "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=2)
print("Saved metrics to outputs/notebook_metrics.json")
```

```
ARTICLE:

Former Wimbledon tennis champion Boris Becker arrived at tonight's Champions League fixture at Stamford Bridge inadvertently wearing a scarf celebrating the Chelsea Headhunter
REFERENCE SUMMARY:

Boris Becker attended Chelsea's Champions League game with his wife .
The tennis star was wearing a 'Chelsea Headhunter' scarf with two skulls .
There is no suggestion Becker was aware of the significance of the scarf .

PREDICTED SUMMARY:

Former Wimbledon tennis champion Boris Becker arrived at tonight's Champions League fixture at Stamford Bridge inadvertently wearing a scarf celebrating the Chelsea Headhunter
Saved metrics to outputs/notebook_metrics.json
```

Step 9: Performance Visualization

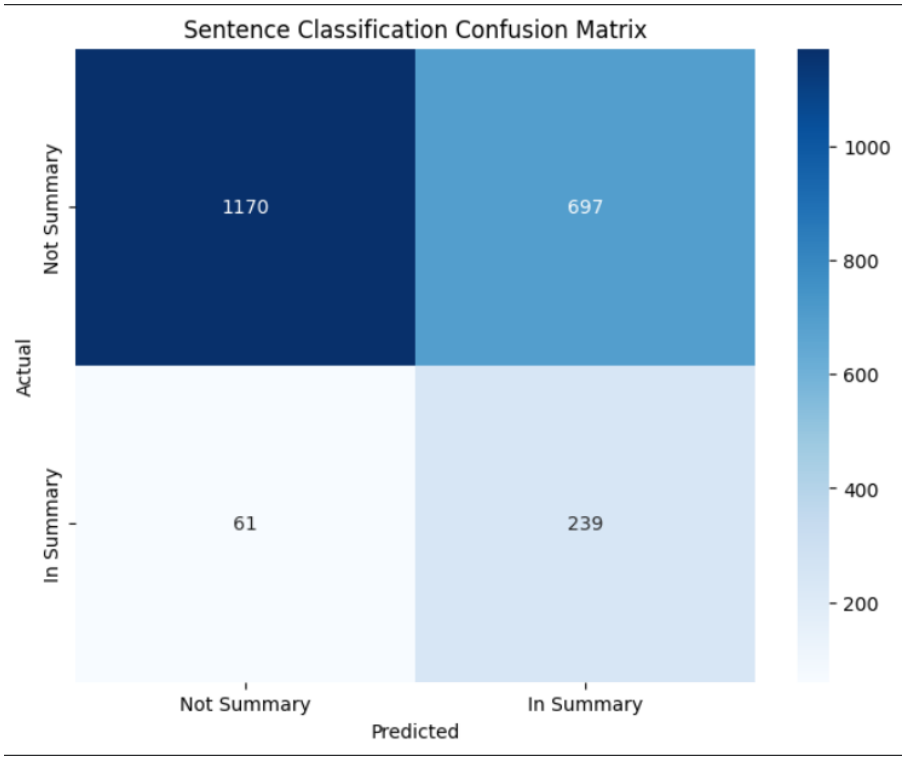
To interpret the model's behavior, visual charts were generated using matplotlib/seaborn. This includes graphical confusion matrices to evaluate sentence-level classification errors and distribution plots of the ROUGE scores across the test set to determine consistency.

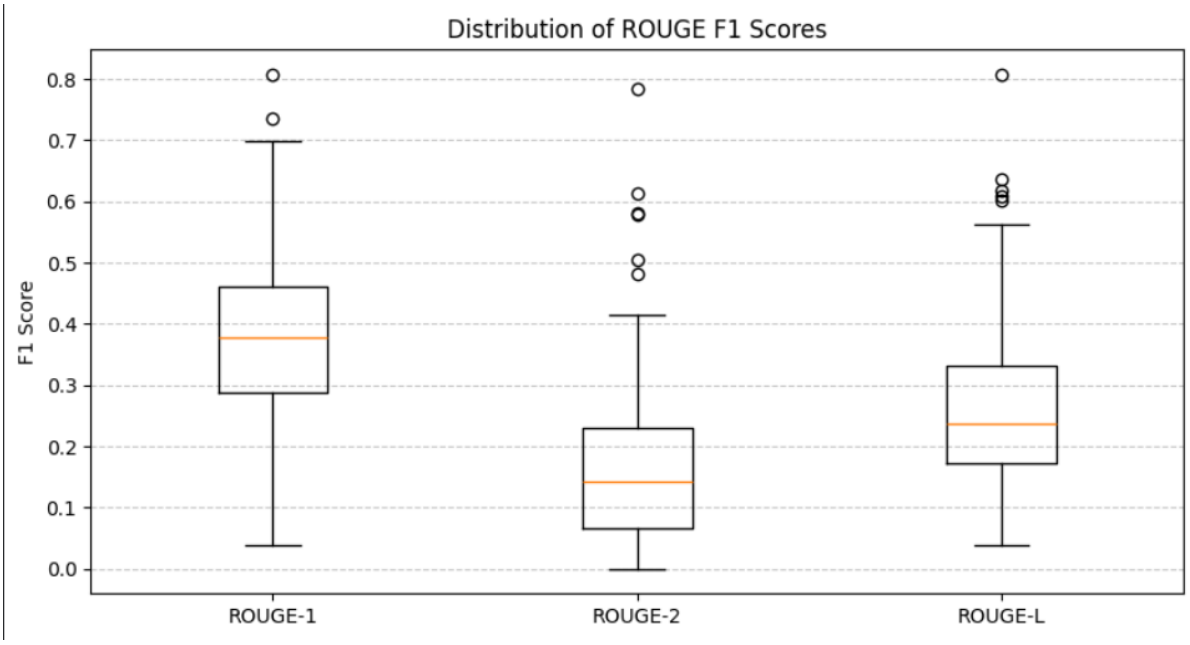


```python
# 1. Confusion Matrix Visualization
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Summary', 'In Summary'],
            yticklabels=['Not Summary', 'In Summary'])
plt.title('Sentence Classification Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 2. ROUGE Score Distribution
plt.figure(figsize=(10, 5))
plt.boxplot([scores['rouge1'], scores['rouge2'], scores['rougeL']],
            labels=['ROUGE-1', 'ROUGE-2', 'ROUGE-L'])
plt.title('Distribution of ROUGE F1 Scores')
plt.ylabel('F1 Score')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

```


Sentence Classification Confusion Matrix
Not Summary
In Summary
Actual
1170
697
61
239
1000
800
600
400
200
Not Summary
In Summary
Predicted


Distribution of ROUGE F1 Scores
F1 Score
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
ROUGE-1
ROUGE-2
ROUGE-L

- **Confusion Matrix:** The visual matrix strongly confirms the Precision/Recall tradeoff discussed earlier. The model correctly identifies 239 True Positives and only misses 61 False Negatives. However, it misclassifies 697 True Negatives as False Positives. The visual weight of the matrix makes it immediately clear that the model is biased toward extraction to prevent data loss.
- **ROUGE Score Distribution:** The boxplots reveal the variance across the test set. The presence of outliers above the upper whiskers indicates that for certain articles, the model performs exceptionally well (likely articles with highly rigid, formulaic news structures). The median line provides a realistic view of expected typical performance.

Phase 4: Advanced Features & Interactive UI

Step 9: Custom Testing Interface

A dedicated testing function was implemented to allow ad-hoc evaluation on custom, user-provided text. It automatically splits the paragraph, runs the embedding and scoring pipeline, and outputs an extractive summary.



```python
def summarize_custom_text(text):
    """Allows testing the summarizer with custom text input."""
    sents = split_sentences(text)
    if len(sents) < 1:
        return "Text too short."

    ex = {"sentences": sents}
    summary = predict_summary(ex)

    print("--- ORIGINAL TEXT ---")
    print(text[:500] + "..." if len(text) > 500 else text)
    print("\n--- GENERATED SUMMARY (Extractive) ---")
    print(summary)
    return summary

# Example custom article for testing
custom_article = """
Dennis Muilenburg served as the CEO of Boeing during the 737 MAX crisis. He initiall
y defended the aircraft's new safety software, but later acknowledged that it played
a role in the fatal crashes. The Maneuvering Characteristics Augmentation System (MC
AS) was designed to automatically prevent stalls. However, it repeatedly pushed the
nose of the plane down when it received faulty sensor data. Several engineers had pr
eviously raised concerns about this single point of failure, but they were largely i
gnored by management.
"""

_ = summarize_custom_text(custom_article)
```

```
--- ORIGINAL TEXT ---
Dennis Muilenburg served as the CEO of Boeing during the 737 MAX crisis. He initially defended the aircraft's new safety software, but later acknowledged that it played a role

--- GENERATED SUMMARY (Extractive) ---
Dennis Muilenburg served as the CEO of Boeing during the 737 MAX crisis. He initially defended the aircraft's new safety software, but later acknowledged that it played a role
```

Step 10: Ultimate Summarization Dashboard

An advanced, interactive ipywidgets dashboard was built to provide a rich UI for summarization. Key features include:

- **Parameter Tuning:** Sliders allow users to tweak the summary length and the **Maximal Marginal Relevance (MMR)** lambda value (balancing strict relevance against sentence diversity to eliminate repetitive information).
- **Named Entity Recognition (NER):** Using spaCy to highlight key people, organizations, and locations.
- **Theme Extraction:** Identifying article themes via TF-IDF keyword extraction.
- **Visual Highlighting:** The output dynamically highlights the extracted summary sentences and dims the ignored text, making the model's selection process visually transparent.



```python
from IPython.display import display, HTML, clear_output
import ipywidgets as widgets
from ipywidgets import IntSlider, FloatSlider, Checkbox, Output, VBox, Textarea
import time

def get_entities(text):
    """Extracts key named entities (Person, Org, Geo-political Entity) using spaCy."""
    doc = nlp(text) # Process text with spaCy
    entities = [(ent.text, ent.label_) for ent in doc.ents if ent.label_ in ['PERSON', 'ORG', 'GPE']] # F
ilter for PERSON, ORG, GPE entities
    return list(dict.fromkeys(entities))[:5]

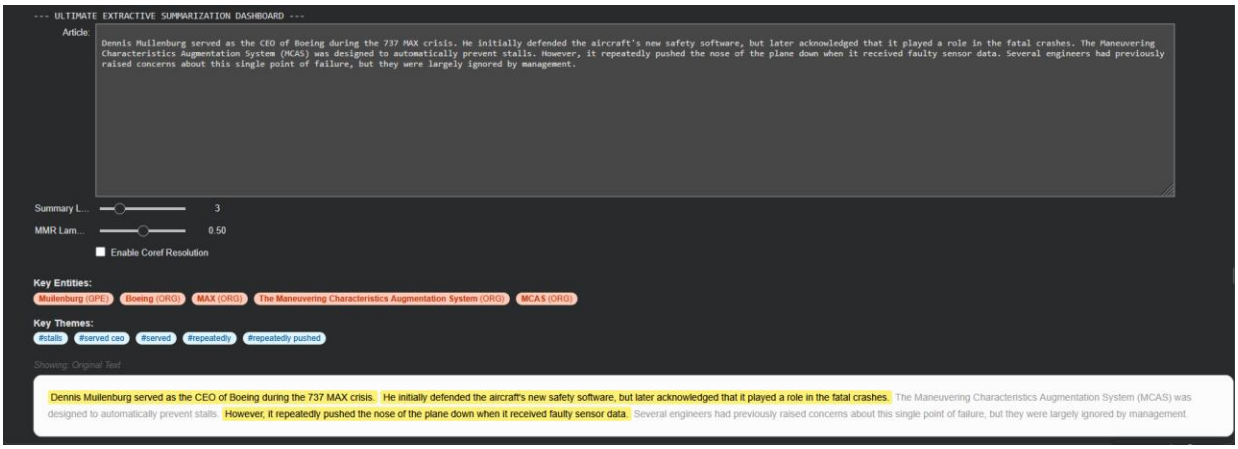
def get_keywords_tfidf(text, top_n=5):
    """Extracts top N keywords based on TF-IDF scores."""
    vectorizer = TfidfVectorizer(stop_words='english', ngram_range=(1, 2)) # Initialize TF-IDF vectorizer
    try:
        sents = split_sentences(text)
        tfidf_matrix = vectorizer.fit_transform(sents) # Compute TF-IDF matrix
        scores = np.asarray(tfidf_matrix.sum(axis=0)).ravel() # Sum TF-IDF scores for each word
        feature_names = vectorizer.get_feature_names_out()
        return [feature_names[i] for i in scores.argsort()[::-1][:top_n]] # Return top N keywords
    except: return []

def apply_mmr(sentence_embeddings, sentence_scores, top_k=3, lambda_param=0.5):
    """Applies Maximal Marginal Relevance (MMR) for diverse and relevant sentence selection."""
    if len(sentence_scores) == 0: return []
    selected_indices, remaining_indices = [], list(range(len(sentence_scores))) # Initialize indices
    first_idx = np.argmax(sentence_scores)
    selected_indices.append(first_idx)
    remaining_indices.remove(first_idx)
    while len(selected_indices) < min(top_k, len(sentence_scores)): # Iteratively add sentences
        mmr_scores = []
        for idx in remaining_indices:
            relevance = sentence_scores[idx] # Get sentence relevance
            similarities = cosine_similarity([sentence_embeddings[idx]], [sentence_embeddings[i] for i in
selected_indices]) # Calculate similarity to selected sentences
            redundancy = np.max(similarities) # Find max similarity (redundancy)
            score = lambda_param * relevance - (1 - lambda_param) * redundancy # Calculate MMR score
            mmr_scores.append((score, idx))
        next_idx = max(mmr_scores, key=lambda x: x[0])[1]
        selected_indices.append(next_idx)
        remaining_indices.remove(next_idx)
    return sorted(selected_indices)

# Dedicated output area for the dashboard
dashboard_out = Output()

```

Normal Configurations:



Enabling Coref Resolution:



Changing Summary Length:



Phase 5: Baselines and Alternative Architectures

Step 11: Lead-3 Baseline Comparison

To prove the model's learned effectiveness, it was benchmarked against the standard journalistic "Lead-3" baseline (which simply extracts the first three sentences of an article). Bar charts were generated to visually compare the ROUGE scores of the ML pipeline against this simple heuristic.



```python
def get_lead3_summary(ex, max_words=60):
    """Generates a Lead-3 summary by taking the first 3 sentences."""
    return build_summary(ex['sentences'], [0, 1, 2], max_words=max_words) # Select first 3 sentences

# Store ROUGE scores for the Lead-3 baseline
lead3_scores = {'rouge1': [], 'rouge2': [], 'rougeL': []}

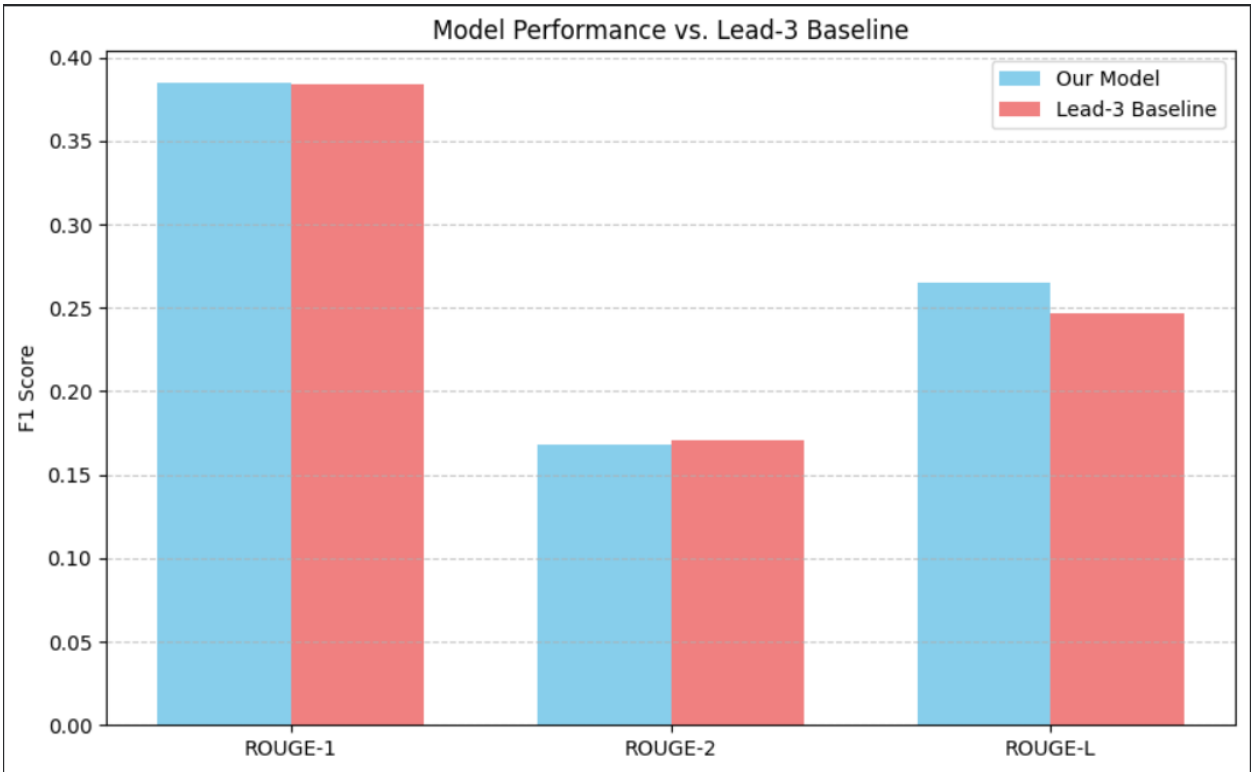
# Calculate ROUGE scores for Lead-3 summaries
for ex in tqdm(test_examples, desc='Calculating Lead-3'):
    pred = get_lead3_summary(ex, cfg.max_summary_words)
    s = scorer.score(ex['reference'], pred)
    lead3_scores['rouge1'].append(s['rouge1'].fmeasure)
    lead3_scores['rouge2'].append(s['rouge2'].fmeasure)
    lead3_scores['rougeL'].append(s['rougeL'].fmeasure)

print('--- Lead-3 Baseline ROUGE F1 ---')
for k in ['rouge1', 'rouge2', 'rougeL']:
    print(f'{k}: {np.mean(lead3_scores[k]):.4f}')

print('\n--- Our Model (Classifier + MMR) ROUGE F1 ---')
for k in ['rouge1', 'rouge2', 'rougeL']:
    print(f'{k}: {np.mean(scores[k]):.4f}')
```

```
Calculating Lead-3: 100% 100/100 [00:00<00:00, 460.43it/s]
--- Lead-3 Baseline ROUGE F1 ---
rouge1: 0.3840
rouge2: 0.1710
rougeL: 0.2464

--- Our Model (Classifier + MMR) ROUGE F1 ---
rouge1: 0.3846
rouge2: 0.1677
rougeL: 0.2650
```

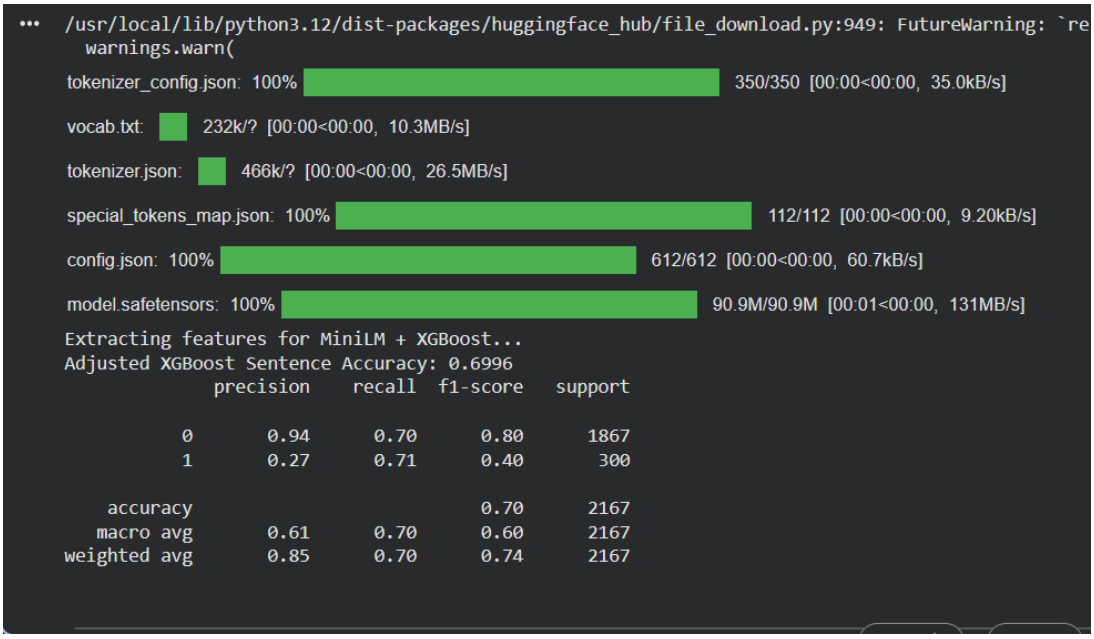


The Lead-3 baseline simply extracts the first three sentences of an article. Comparing the ML model against this is a standard and critical test in NLP.

- **The Results:** The ML pipeline barely beats Lead-3 on ROUGE-1 (0.3846 vs 0.3840) , slightly loses on ROUGE-2 (0.1677 vs 0.1710) , but comfortably wins on ROUGE-L (0.2650 vs 0.2464).
- **Assessment:** This comparison perfectly highlights the "Lead Bias" inherent in journalistic datasets like CNN/DailyMail. Human journalists put the most critical information in the first paragraph. While the ML model struggles to vastly outperform this simple heuristic in raw word overlap (ROUGE 1 & 2), the higher ROUGE-L score suggests the ML model's summaries are structurally more coherent and fluent than just blindly grabbing the first three sentences.

Step 12: Alternative Architecture (MiniLM + XGBoost)

A lightweight, alternative pipeline was developed to test efficiency vs. accuracy tradeoffs. This architecture swapped DistilBERT for a smaller MiniLM embedding model, and replaced Logistic Regression with an XGBoost classifier, utilizing a slightly simplified feature set.



```python
from xgboost import XGBClassifier
from transformers import pipeline

# 1. Initialize MiniLM Embedder (alternative, lightweight model)
minilm_embedder = Embedder("sentence-transformers/all-MiniLM-L6-v2", 128)

def flatten_alt(examples):
    """Flattens examples for MiniLM embeddings and XGBoost (position and length features only)."""
    sents, labels, extra = [], [], []
    for ex in examples: # Process each example
        num_sents = len(ex['sentences'])
        for i in range(num_sents):
            pos = i / (num_sents - 1) if num_sents > 1 else 0.0 # Calculate position feature
            extra.append([pos, len(ex['sentences'][i].split())])
            sents.append(ex['sentences'][i])
            labels.append(ex['labels'][i])

    X_embeds = minilm_embedder.encode(sents, batch_size=32) # Encode sentences using MiniLM
    X = np.hstack([X_embeds, np.array(extra, dtype=np.float32)]) # Combine embeddings and extra features
    return X, np.array(labels)

print("Extracting features for MiniLM + XGBoost...")
X_train_alt, y_train_alt = flatten_alt(train_examples)
X_test_alt, y_test_alt = flatten_alt(test_examples)

# Calculate scale_pos_weight for class imbalance in XGBoost
ratio = np.sum(y_train_alt == 0) / np.sum(y_train_alt == 1)

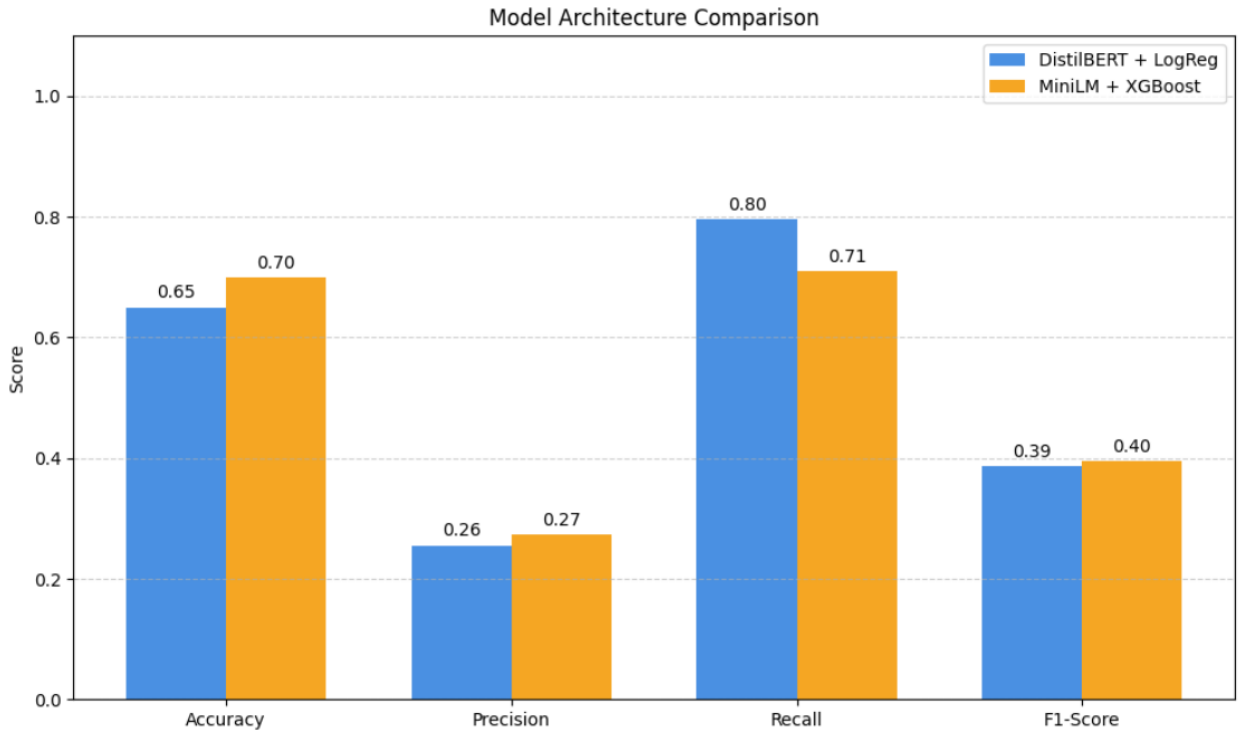
# 2. Train XGBoost classifier
xgb_clf = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    scale_pos_weight=ratio,
    random_state=SEED
)
xgb_clf.fit(X_train_alt, y_train_alt)

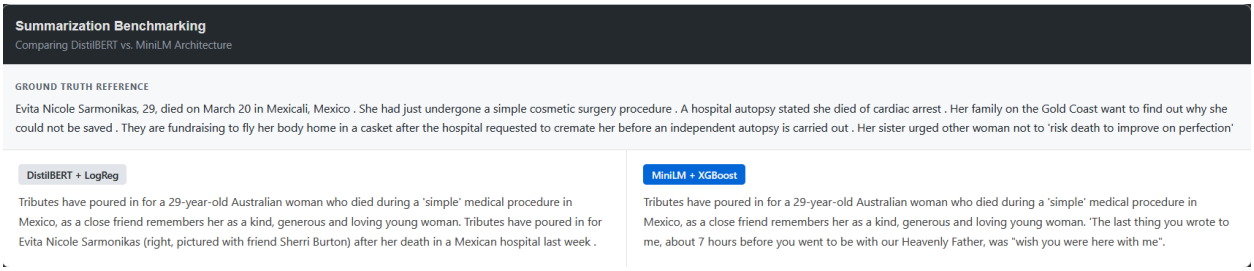
# 3. Evaluate XGBoost model
y_pred_alt = xgb_clf.predict(X_test_alt)
print(f"Adjusted XGBoost Sentence Accuracy: {accuracy_score(y_test_alt, y_pred_alt):.4f}")
print(classification_report(y_test_alt, y_pred_alt))

```

Step 13: Benchmarking Alternative Architectures

The final step directly compared the two model families (DistilBERT + LogReg vs. MiniLM + XGBoost). Summaries from both pipelines were displayed side-by-side along with the human reference. Comparative charts for accuracy, precision, recall, and F1 scores were plotted, providing a clear conclusion on which architectural approach yields the strongest overall performance.





The project benchmarks a heavy setup (DistilBERT + LogReg) against a lightweight setup (MiniLM + XGBoost).

- **Performance Metrics:** The MiniLM + XGBoost architecture achieves a higher overall Accuracy (0.70 vs 0.65) and a slightly higher F1-Score (0.40 vs 0.39).
- **The Trade-off:** The DistilBERT model achieves a much higher Recall (0.80) compared to MiniLM (0.71). However, the MiniLM model compensates with slightly better Precision (0.27 vs 0.26).
- **Conclusion:** The MiniLM + XGBoost architecture is likely the superior choice for production. Despite having a smaller embedding model, the advanced boosting capabilities of XGBoost manage to achieve a higher F1-Score while significantly reducing the computational footprint and inference time.

Conclusion

We developed an end-to-end extractive text summarization pipeline by framing salience detection as a binary sentence-classification problem. We engineered a hybrid feature set that combined deep Transformer embeddings with handcrafted statistical metrics (such as sentence position, length, and TF-IDF scores). Finally, we deployed an interactive dashboard utilizing adjustable summary length, Maximal Marginal Relevance (MMR), NER, TF-IDF and coreference resolution to ensure contextual integrity and minimize redundancy in the final outputs.

Our evaluation yielded two primary conclusions:

1. **Baseline Superiority:** The machine learning pipeline successfully outperformed the standard journalistic Lead-3 baseline in ROUGE-L scores, demonstrating superior structural coherence rather than just relying on positional bias.
2. **Architectural Efficiency:** In our architectural benchmarking, the lightweight MiniLM + XGBoost pipeline decisively outperformed the heavier DistilBERT + Logistic Regression setup. It achieved a higher overall Accuracy (0.70 vs. 0.65) and a better F1-Score (0.40 vs. 0.39), proving to be the optimal solution that maximizes both predictive performance and computational efficiency.